# Programmation FPGA avec Eclat : comparaison de style et analyses des performances

Dahmane Ilan
APR/LIP6 et IRILL

1 Juin 2026 - 31 Juillet 2026

**Tuteur universitaire :**
Eugene Asarin
Giulio Manzonetto

**Tuteur de stage :**
Loïc Sylvestre
Emmanuel Chailloux

# Sommaire

**1 Introduction** **3**

**2 FPGA et programmation** **5**

**3 Éxperimentations** **8**
3.1 Processeur LC-3 . . . . . 8
3.2 Première implantation de LC3 en Eclat . . . . . 9
3.3 Résultats des benchmarks . . . . . 9

**4 Conclusion** **10**

# 1 Introduction

Dans l'industrie informatique, les processeurs généralistes (CPU) constituent depuis plusieurs décennies la solution dominante pour l'exécution des programmes. Leur architecture est conçue pour offrir une grande flexibilité et pouvoir traiter une grande variété de tâches. Cependant, cette généralité limite leur efficacité lorsqu'il s'agit d'exploiter le parallélisme massif présent dans certaines applications.

Pour répondre à certains besoins spécifiques, les GPU (Graphics Processing Units) ont été développés initialement pour accélérer le rendu graphique. Leur architecture massivement parallèle les rend particulièrement efficaces pour les applications reposant sur de nombreuses opérations similaires appliquées sur de grands ensembles de données, comme le traitement d'image ou l'intelligence artificielle.

Cependant, toutes les applications ne se prêtent pas naturellement à ce modèle de calcul. Certaines nécessitent des architectures spécialisées capables d'exploiter un parallélisme différent, avec des contraintes fortes en termes de latence, de consommation énergétique ou de traitement temps réel. C'est dans ce contexte que les FPGA (Field Programmable Gate Array) constituent une solution intéressante grâce à leur capacité de reconfiguration [1] et leur aptitude à implémenter des architectures matérielles spécialisées.

Cependant, leur programmation reste un domaine complexe nécessitant traditionnellement une description matérielle bas niveau. Le langage Eclat [2] s'inscrit dans cette problématique en proposant une approche permettant de décrire des calculs matériels avec un niveau d'abstraction supérieur tout en conservant une traduction efficace vers des architectures FPGA.

Si les langages de plus haut niveau comme Eclat permettent de simplifier la programmation des FPGA, ils soulèvent néanmoins une question importante. Les concepteurs matériels raisonnent traditionnellement en termes de registres, de portes logiques, de mémoires et de cycles d'horloge, tandis que les programmeurs sont habitués à manipuler des abstractions de plus haut niveau, des structures de données complexes et différents paradigmes de programmation. Il devient alors nécessaire de déterminer quels styles de programmation permettent de concilier expressivité, sûreté et efficacité du matériel généré.

Dans quelle mesure les styles de programmation fonctionnel et impératif proposés par Eclat permettent-ils de décrire efficacement des algorithmes réalistes tout en générant des architectures matérielles performantes ?

Pour répondre à cette problématique, ce stage consiste à comparer les approches fonctionnelle et impérative au travers de l'implémentations d'un processeur LC-3 [1], étudié à l'université de Paris-Cité durant le cours de Circuits et Architecture enseigné par le professeur Adrien Guatto. Le processeur LC-3 constitue un excellent candidat pour cette évaluation grâce à son compromis entre simplicité et représentativité. Son architecture volontairement réduite

1. Le LC-3 est un processeur pédagogique conçu par Yale N.Patt et Sanjay J.Patel. Introduit dans leur ouvrage Introduction to Computing Systems afin d'enseigner les principes fondamentaux de l'architecture des ordinateurs.

permet une compréhension complète de son fonctionnement, tout en mettant en œuvre de nombreux mécanismes caractéristiques des processeurs modernes. Il sollicite ainsi plusieurs fonctionnalités importantes d'Eclat, telles que la gestion de l'état, des registres, de la mémoire et des structures de données. Cette complexité maîtrisée en fait un support pertinent pour tester un large panel des fonctionnalités du langage et comparer différentes approches d'implémentation. De plus, l'utilisation d'un même processeur comme base d'expérimentation permet de conserver un comportement fonctionnel identique entre les différentes versions, tout en observant l'impact du style de programmation choisi sur l'architecture matérielle générée, les performances obtenues et les ressources utilisées. Les deux styles sont évalués selon différents critères tels que la facilité d'écriture, les performances obtenues après synthèse et l'occupation des ressources matérielles sur FPGA.

## 2 FPGA et programmation

Les FPGA représentent une évolution majeure dans le domaine de la conception matérielle. Contrairement aux circuits intégrés spécifiques (ASIC), dont la fabrication nécessite des coûts de conception initiaux très importants, les FPGA permettent de configurer après fabrication une architecture matérielle adaptée à un besoin particulier. Cette flexibilité réduit considérablement les coûts et les délais de développement, notamment pour les systèmes nécessitant des évolutions fréquentes ou des productions en volumes limités.

La particularité principale des FPGA réside dans leur capacité de reconfiguration. Ils permettent d'exploiter un parallélisme spatial en instanciant directement plusieurs unités de calcul matérielles. Cette caractéristique en fait des accélérateurs matériels particulièrement adaptés à certaines applications industrielles nécessitant de fortes performances, comme le traitement du signal, l'intelligence artificielle, le calcul scientifique ou encore les systèmes embarqués temps réel.

Les FPGA sont composés de bloc logique reprogrammable permettant cette forte modularité, (voir Figure 1).

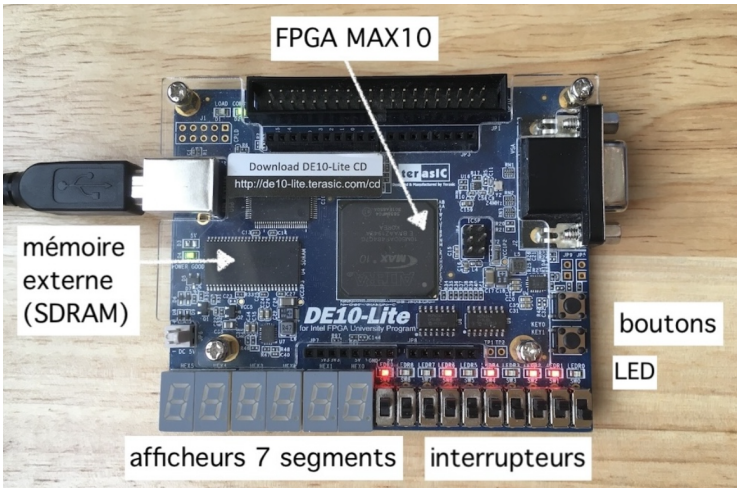



FIGURE 1 – Schéma d'un FPGA : Intel DE-10 lite

Dans le coeur d'une puce FPGA se trouve les blocs logique reprogrammable (voir Figure 2), qui permet toute l'intérêt suscité par ces cartes. Elles sont composé d'une ou plusieurs LUT [2] ainsi qu'un registre qui permet de sauvegarder les états entre chaque cycle d'horloge, sans cela, le circuit serait purement combinatoire.

Plusieurs notions seront utile pour comprendre l'analyse des benchmarks. Nous commencons avec l'horloge et ses notions associés. Un cycle permet de

2. Look-Up Tables : une petite mémoire permettant de réaliser n'importe quelle fonction logique de quelques entrées.

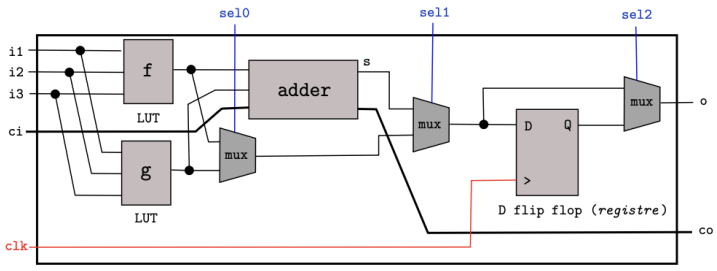



FIGURE 2 – Structure d'un élément logique avec additionneur intégré

faire passer l'éléctricité dans tout le circuit de la puce, accompagné de cela, il y a aussi les front d'horloge qui permettent de mettre à jour toutes les entrées/sorties (registres) du circuit après un cycle.

Toutes puces éléctronique possède une fréquence. Celle de la carte DE-10 lite est de 50MHz, cela signifie que la carte peut exécuter 50 millions de cycles par secondes et donc chaque cycle dure 20ns (1/fréquence). Le circuit va donc se mettre à jour 50 millions de fois par seconde.

Un programme peut être aussi mesuré par son rapport entre la fréquence sur la carte sur laquelle il tourne et le nombre de cycle dont il à besoin pour finir. Prenons un programme A qui prend 5000 cycles avec une fréquence de 50MHz, le programme mettra donc 500 chronosecondes à s'exécuter (500/fréquence).

Cependant, l'exploitation de ces architectures nécessite traditionnellement l'utilisation de langages de description matérielle (HDL), tels que le VHDL ou le Verilog. Contrairement aux langages de programmation classiques, ces langages ne décrivent pas une succession d'instructions exécutées par un processeur, mais une architecture matérielle composée de composants logiques fonctionnant potentiellement en parallèle. Cette approche offre un contrôle très fin sur l'implémentation matérielle, mais introduit également une complexité importante et nécessite des connaissances spécifiques en conception numérique. L'objectif est donc de proposer des outils permettant de rapprocher la programmation FPGA des méthodes utilisées en programmation logicielle classique, tout en conservant un contrôle suffisant sur l'architecture générée..

Un des exemples les plus connus est HLS (High-level Synthesis)[3] afin de se concentrer sur l'aspect algorithmique et n'ont pas sur l'architecture du circuit. Nous pouvons notamment écrire dans un sous ensemble de C/C++ (pointeur et allocation dynamique parfois exclus) afin que l'outil de synthèse de haut niveau parse votre programme et le synthétise en HDL. Mais cette approche à une limite. Le contrôle de l'architecture par le concepteur du programme est grandement limité et est délégué à l'outil de synthèse (le parallélisme est inférer etc... creuser un peu plus).

C'est dans cette perspective que s'inscrit Eclat. Ce langage propose un paradigme de programmation plus proche des langages logiciels traditionnels (OCaml-like) tout en conservant la possibilité de générer une architecture ma-

térielle efficace. Le compilateur Eclat permet ainsi de traduire les programmes écrits dans ce langage en une description matérielle VHDL synthétisable, pouvant ensuite être implémentée sur un FPGA.

# 3 Éxperimentations

## 3.1 Processeur LC-3

Pour répondre à notre question, nous allons reprendre un ancien projet de mon premier semestre de master. Ce projet portait sur la réalisation d'un processeur à but éducatif nommé LC-3. C'est un processeur très rudimentaire, comprenant d'environ 15 instructions de base. Ces dernières se décomposent en trois familles : les instructions arithmétiques et logiques, les instructions de chargement et rangement et les instructions de branchement (appelées aussi instructions de saut ou encore instructions de contrôle).

Les instructions arithmétiques et logiques du LC-3 sont au nombre de trois. Le LC-3 possède une instruction ADD pour l'addition, une instruction AND pour le et logique bit à bit et une instruction NOT pour la négation bit à bit. Le résultat de ces trois opérations est toujours placé dans un registre. Les deux opérandes peuvent être soit les contenus de deux registres soit le contenu d'un registre et une constante pour ADD et AND.

Les instructions de chargement et rangement comprennent des instructions (avec des noms commencent par LD) permettant de charger un registre avec une valeur et des instructions (avec des noms commencent par ST) permettant de ranger en mémoire le contenu d'un registre. Ces instructions se différencient par leurs modes d'adressage qui peut être immédiat, direct, indirect ou relatif. Les instructions de chargement sont LD, LDI, LDR, LEA et les instructions de rangement sont ST, STI et STR

Les instructions de branchement comprennent les deux instructions de saut BR et JMP, les deux instructions d'appel de sous-routine JSR et JSRR, une instruction TRAP d'appel système (pas implementé), une instruction RET de retour de sous-routine et une instruction RTI de retour d'interruption (pas implementé).

De plus, le micro-processeur LC-3 dispose de 8 registres généraux 16 bits appelés R0...R7 et comme tous les micro-processeurs, le LC-3 dispose d'un compteur de programme PC et d'un registre d'instruction IR qui sont tous les deux des registres 16 bits.

Les indicateurs N, Z et P sont des registres 1 bit. Ils sont positionnés dès qu'une nouvelle valeur est mise dans un des registres généraux R0...R7. Ceci a lieu lors de l'exécution d'une instruction logique (NOT et AND), arithmétique (ADD) ou d'une instruction de chargement (LD, LDI, LDR et LEA). Ils indiquent respectivement si cette nouvelle valeur est négative (n), nulle (z) et positive (p). Ces indicateurs sont utilisés par l'instruction de branchement conditionnel BR pour savoir si le branchement doit être pris ou non.

La mémoire du LC-3 est organisée par mots de 16 bits. L'adressage du LC-3 est également de 16 bits. La mémoire du LC-3 est donc formée de 216 mots de 16 bits, c'est-à-dire 128 KiB avec des adresses de `0x0000` à `0XFFFF` (voir Figure 3).

## 3.2 Première implantation de LC3 en Eclat

Pour la première implémentation en Eclat du processeur, j'ai choisi de privilégier une approche impérative, plus proche des langages de programmation classiques et plus pédagogique pour prendre en main Eclat. Les registres ainsi que la mémoire ont été représentés par des tableaux, qui constituent la principale structure de données mutable du langage. Les modifications sont effectuées en place, c'est-à-dire par réécriture directe des cases mémoire concernées, sans recopier l'ensemble de la structure. Le tableau représentant la mémoire contient des valeurs du type somme value, capable de représenter les différents éléments susceptibles d'être stockés en mémoire, tels qu'une instruction, une adresse, un caractère ou une constante. Cette première implémentation est visible en annexe (le sera bientôt...). Plusieurs problèmes se sont posés durant l'implémentations de ce premier processeur.

## 3.3 Résultats des benchmarks

J'ai commencé par allumer simultannémament un paquet de leds afin de comprendre la chaîne de compilation d'un langage de description matériel jusqu'à la production du .sof qui décrit le circuit et permet de flash sur la carte le circuit synthétiser par eclat.

- Parler du nombre de bloc logique utilisé pour les deux synthèse - Parler du temps en cycle - Parler de la facilité d'écriture - Insérer des benchs avec un programme équivalent mais synthétisé avec la vm OCaml en Eclat

# 4 Conclusion

Nous pouvons voir que les deux approches ont leur qualités et leur défauts, d'un côté nous avons de la performance temporelle pour le style fonctionnel qui permet de réduire les couts d'accès du à l'abscence d'accès concurrent (qui coute 1 cycle par accès) mais d'un autre côté nous avons une explosion du nombre de blocs logiques programmables utilisé en raison des nombreuses copies de bus de fils etc... De l'autre nous avons l'exacte opposé, en terme de complexité pire mais qui en terme de taille de circuit et de blocs logiques programmables est bien supérieur et permet de faire des programme bien plus gros. Une seconde explication passe par la logique d'implémentation des structure de données des deux styles. Les tableaux sont dans la BRAM (je crois car il y a aussi la SDRAM mais c'est une mémoire externe et c'est pour les grosses donneés non ?) de la carte ce qui ne limite pas sa taille tandis que les vecteurs sont des suites de blocs logiques programmables ce qui limite sa taille à 128kb (voir photo ou plutot donner une équivalent en terme de nombre d'entiers 32 bits, plus clair) hypothétique en attendant de finir les benchmarks et donc la conclusion.....

## Références


[1] Olivier Sentieys and Arnaud Tisserand. Architectures reconfigurables FPGA. In Techniques de l'Ingénieur, editor, *Technologies logicielles Architectures des systèmes*, volume H 1 196, pages 1–22. Techniques de l'Ingénieur, August 2012.

[2] Loïc Sylvestre. Programmation sûre, expressive et efficace de circuits fpga. 2025.

[3] Daouda Diakité. Synthèse de haut niveau sur fpga pour problèmes inverses : application à la tomographie et à la radioastronomie. 2023.

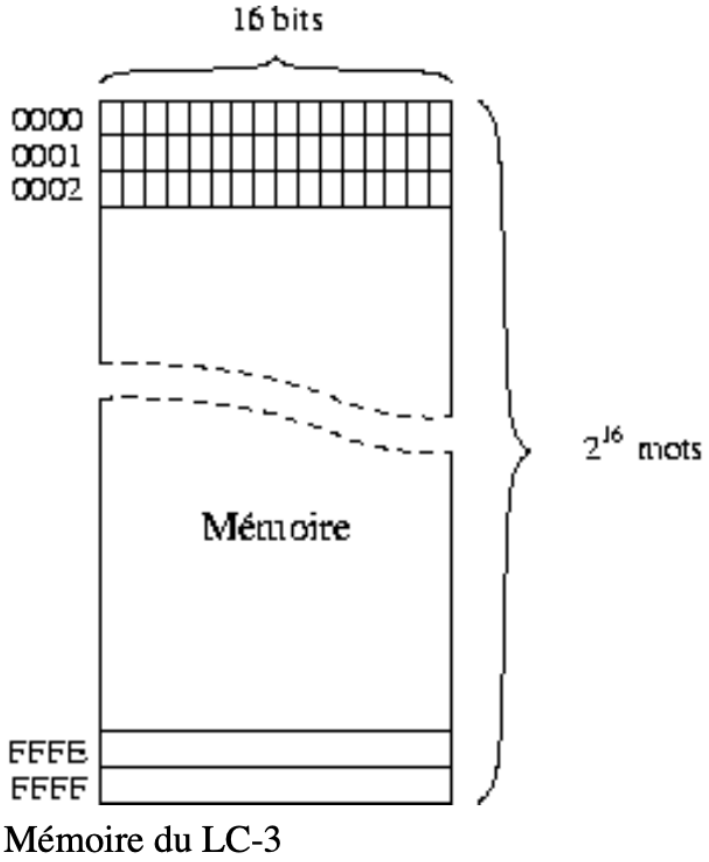



FIGURE 3 – Représentation de la mémoire LC-3

| Syntaxe | Action | nzp | Codage | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Op-code | | | | Arguments | | | | | | | | | | | |
| | | | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| NOT DR,SR | DR ← not SR | * | 1 | 0 | 0 | 1 | DR | | | SR | | | 1 | 1 | 1 | 1 | 1 | 1 |
| ADD DR,SR1,SR2 | DR ← SR1 + SR2 | * | 0 | 0 | 0 | 1 | DR | | | SR1 | | | 0 | 0 | 0 | SR2 | | |
| ADD DR,SR1,Imm5 | DR ← SR1 + SEXT(Imm5) | * | 0 | 0 | 0 | 1 | DR | | | SR1 | | | 1 | Imm5 | | | | |
| AND DR,SR1,SR2 | DR ← SR1 and SR2 | * | 0 | 1 | 0 | 1 | DR | | | SR1 | | | 0 | 0 | 0 | SR2 | | |
| AND DR,SR1,Imm5 | DR ← SR1 and SEXT(Imm5) | * | 0 | 1 | 0 | 1 | DR | | | SR1 | | | 1 | Imm5 | | | | |
| LEA DR,label | DR ← PC + SEXT(PCoffset9) | * | 1 | 1 | 1 | 0 | DR | | | PCoffset9 | | | | | | | | |
| LD DR,label | DR ← mem[PC + SEXT(PCoffset9)] | * | 0 | 0 | 1 | 0 | DR | | | PCoffset9 | | | | | | | | |
| ST SR,label | mem[PC + SEXT(PCoffset9)] ← SR | | 0 | 0 | 1 | 1 | SR | | | PCoffset9 | | | | | | | | |
| LDR DR,BaseR,Offset6 | DR ← mem[BaseR + SEXT(Offset6)] | * | 0 | 1 | 1 | 0 | DR | | | BaseR | | | Offset6 | | | | | |
| STR SR,BaseR,Offset6 | mem[BaseR + SEXT(Offset6)] ← SR | | 0 | 1 | 1 | 1 | SR | | | BaseR | | | Offset6 | | | | | |
| LDI DR,label | DR ← mem[mem[PC + SEXT(PCoffset9)]] | * | 1 | 0 | 1 | 0 | DR | | | PCoffset9 | | | | | | | | |
| STI SR,label | mem[mem[PC + SEXT(PCoffset9)]] ← SR | | 1 | 0 | 1 | 1 | SR | | | PCoffset9 | | | | | | | | |
| BR[n][z][p] label | Si (cond) PC ← PC + SEXT(PCoffset9) | | 0 | 0 | 0 | 0 | n | | z | p | PCoffset9 | | | | | | | |
| NOP | No Operation | | 0 | 0 | 0 | 0 | 0 | | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| JMP BaseR | PC ← BaseR | | 1 | 1 | 0 | 0 | 0 | 0 | 0 | BaseR | | | 0 | 0 | 0 | 0 | 0 | 0 |
| RET (≡ JMP R7) | PC ← R7 | | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| JSR label | R7 ← PC; PC ← PC + SEXT(PCoffset11) | | 0 | 1 | 0 | 0 | 1 | PCoffset11 | | | | | | | | | | |
| JSRR BaseR | R7 ← PC; PC ← BaseR | | 0 | 1 | 0 | 0 | 0 | | 0 | 0 | BaseR | | 0 | 0 | 0 | 0 | 0 | 0 |
| RTI | cf. interruptions | | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| TRAP Trapvect8 | R7 ← PC; PC ← mem[Trapvect8] | | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | Trapvect8 | | | | | | | |
| Réservé | | | 1 | 1 | 0 | 1 | | | | | | | | | | | | |

FIGURE 4 – Ensemble du jeu d'instruction du LC-3 (ps : les * indiquent si l'instruction mets à jour les flags NZP)